МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №5

Разработка веб-приложения на Spring Boot.

Основы Spring Security

Выполнил: студент группы ___БВТ2402___

Голиков Михаил Вячеславович

Руководитель: _Мосева Марина Сергеевна_

Москва, 2077

Цель работы

Изучить основы обеспечения безопасности в Spring Boot-приложении: научиться настраивать аутентификацию и авторизацию пользователей, ограничивать доступ к ресурсам приложения, работать с ролями и механизмами защиты запросов, а также познакомиться с основными подходами к хранению состояния пользователя в защищенной системе.

Индивидуальное задание

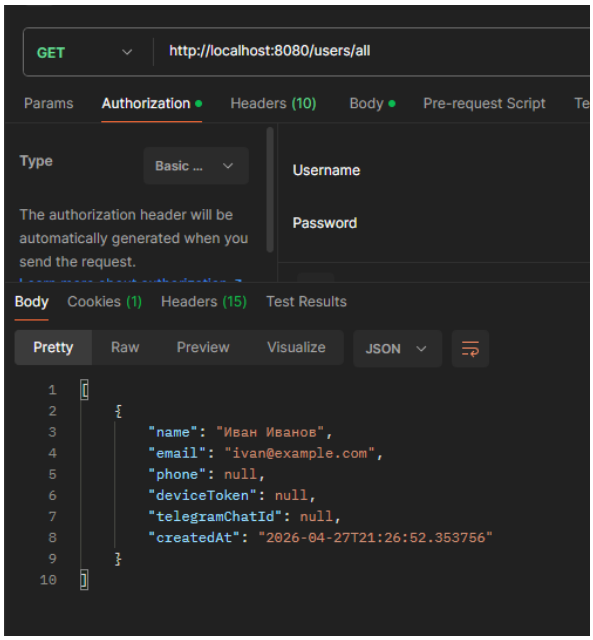
На основе выполненной лабораторной работы №4 (система управления уведомлениями) добавьте в приложение поддержку безопасности с использованием Spring Security. Реализуйте аутентификацию и авторизацию пользователей, заменив имитацию пользователей на реальную защиту на основе базы данных. Расширьте сущность User, добавив поля password и role. Обеспечьте хранение паролей в зашифрованном виде с помощью BCryptPasswordEncoder. Реализуйте регистрацию пользователей с проверкой уникальности email и автоматическим назначением роли ROLE_USER. Создайте отдельный endpoint для регистрации администратора с фиксированным email (например, admin@mail.ru). Настройте доступ к URL в зависимости от ролей: /auth/** – открыт для всех, /admin/** – только для ROLE_ADMIN, /users/** и /notifications/** – для аутентифицированных пользователей. Добавьте поддержку HTTP Basic для тестирования, а затем реализуйте stateless-аутентификацию с использованием JWT, включая генерацию токена при логине, фильтр для проверки JWT и выборочный отказ от сессий. Реализуйте метод extractUsername() в JwtService и проверку срока действия токена. Добавьте обработку ошибок при неверном логине или пароле с возвратом корректного HTTP-статуса. Защитите метод удаления пользователя с помощью @PreAuthorize("hasRole('ADMIN')"). Продемонстрируйте работу всех эндпоинтов через Postman: регистрацию, логин, доступ к закрытым ресурсам с Basic Auth и JWT, а также разграничение доступа по ролям.

Убедитесь, что при попытке обращения без авторизации или с недостаточными правами возвращаются ошибки 401 или 403.

Основная часть

В ходе выполнения лабораторной работы были последовательно реализованы все этапы для изучения безопасности в Spring Boot с использованием Spring Security, JWT и интеграции с базой данных PostgreSQL.

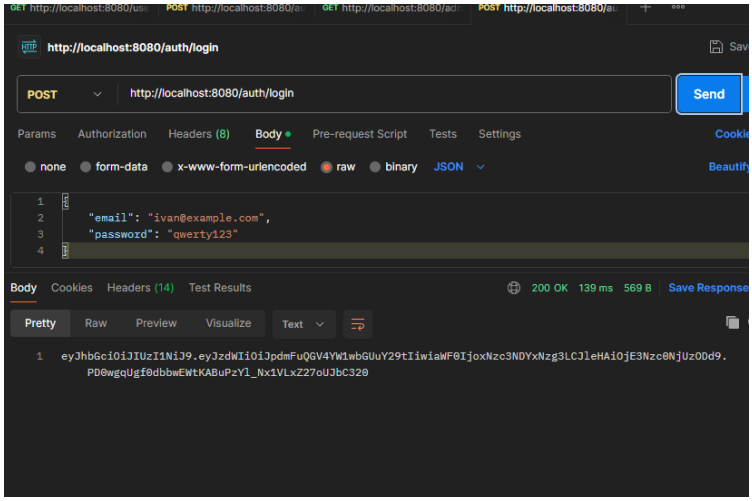
После добавления зависимости spring-boot-starter-security и запуска приложения все URL стали закрытыми. Для проверки был отправлен GET-запрос к эндпоинту /users/all. На изображении 1 показано, что сервер возвращает ошибку доступа, так как пользователь не аутентифицирован. Это демонстрирует, что Spring Security автоматически защитил все endpoints приложения по умолчанию.



Элемент 1 — Отправка ответа только с авторизацией

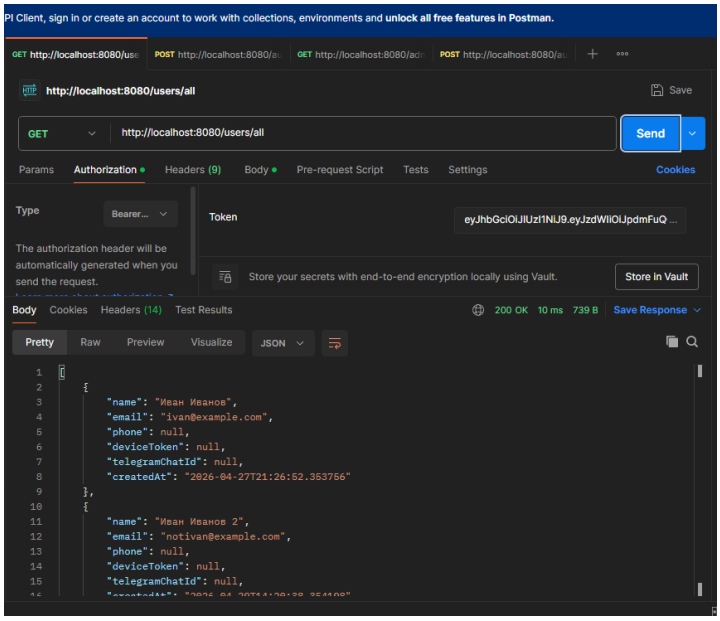
Для аутентификации пользователя был реализован endpoint /auth/login, который принимает email и пароль, проверяет их через AuthenticationManager и при успешной проверке возвращает JWT-токен. На изображении 2 показан успешный запрос логина. В ответе сервер

возвращает сгенерированный JWT-токен, который клиент должен использовать для последующих запросов.



Элемент 2 — Получение JWT для авторизации

После получения JWT-токена был выполнен GET-запрос к /users/all с добавлением заголовка Authorization: Bearer <токен>. На изображении 3 показано, что сервер успешно распознал токен, аутентифицировал пользователя и вернул список всех пользователей. Это подтверждает корректную работу JWT-фильтра, который извлекает токен, проверяет его и устанавливает контекст безопасности.



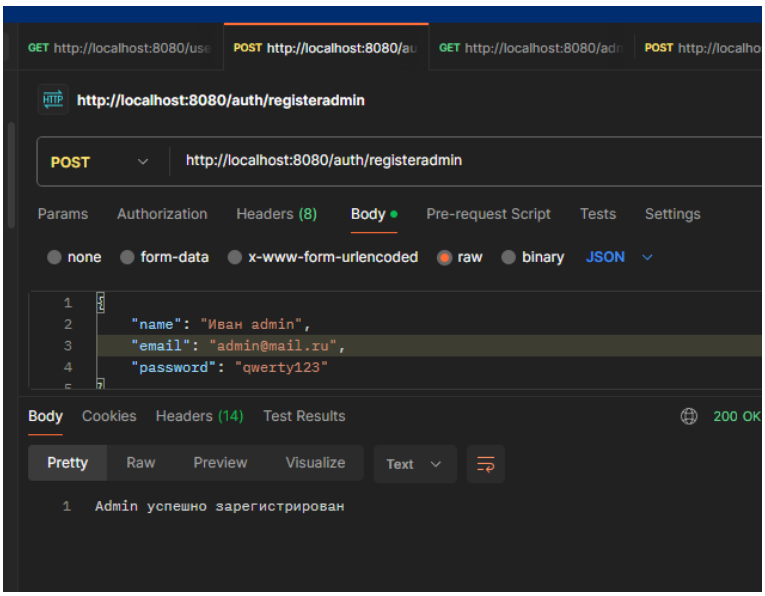
Элемент 3 — Использование JWT для авторизации

Был добавлен специальный endpoint /auth/registeradmin для регистрации администраторов. В листинге под номером 4 представлен код этого метода. Он проверяет, совпадает ли email пользователя с заранее заданным административным email (admin@mail.ru). Если условие выполняется, вызывается метод registerAdmin(), который назначает пользователю роль ROLE_ADMIN. Такая реализация позволяет контролировать создание учётных записей с повышенными привилегиями.

```
@PostMapping("/registeradmin")
public String registeradmin(@RequestBody @Valid RegisterRequest request) {
    String adminEmail = "admin@mail.ru";
    try {
        if (request.getEmail().equals(adminEmail)) {
            authService.registerAdmin(request);
            return "Admin успешно зарегистрирован";
        }
        return "Ошибка регистрации";
    }
    catch (RuntimeException e){
        return "Ошибка регистрации пользователя";
    }
}
```

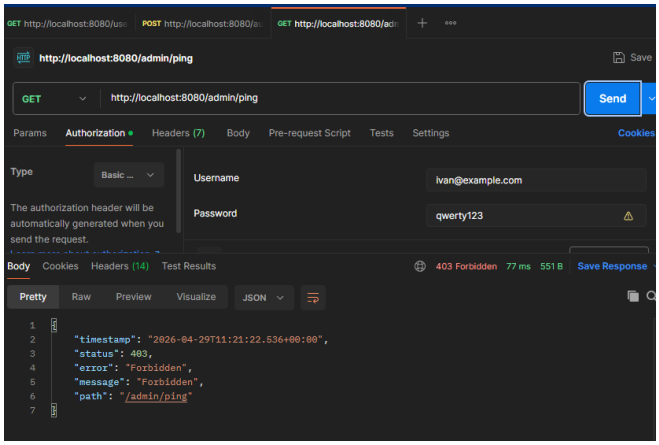
Элемент 4 — Endpoint для регистрации администратора

На изображении 5 показан успешный запрос к endpoint регистрации администратора. При отправке email admin@mail.ru сервер возвращает сообщение об успешной регистрации. В базе данных этому пользователю автоматически присваивается роль ROLE_ADMIN, что позволяет ему в дальнейшем получать доступ к административным функциям.



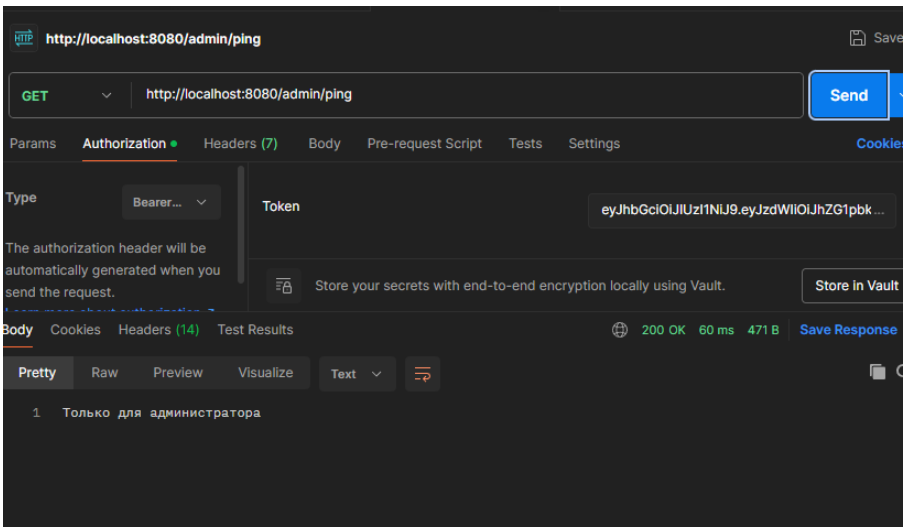
Элемент 5 — Использование endpoint для регистрации администратора

Для проверки разграничения ролей был создан административный endpoint /admin/ping. На изображении 6 показана попытка обычного пользователя (с ролью ROLE_USER) обратиться к этому URL. Сервер возвращает ошибку 403 Forbidden, так как у пользователя недостаточно прав для доступа к административному ресурсу. Это демонстрирует корректную работу правил авторизации, настроенных в SecurityConfig.



Элемент 6 — Из-за отсутствия прав /admin/ping недоступен

После регистрации и аутентификации администратора был выполнен запрос к /admin/ping. На изображении 7 показано, что администратор успешно получает доступ к ресурсу. Сервер возвращает сообщение "Только для администратора", подтверждая, что роль ROLE_ADMIN даёт права на доступ ко всем административным endpoints.



Элемент 7 — Использование прав admin для секретных команд

В сервисе AuthService была добавлена проверка уникальности email при регистрации. В листинге под номером 8 представлен обновлённый метод register(). Перед сохранением нового пользователя выполняется поиск существующего пользователя с таким же email. Если запись найдена, выбрасывается исключение RuntimeException. Это предотвращает создание дублирующихся учётных записей и обеспечивает целостность данных.

```
public void register(RegisterRequest request) {
    if (userRepository.findByEmail(request.getEmail()).isPresent()){
        throw new RuntimeException("Пользователь с email " +
request.getEmail() + " уже существует");
    }
    User user = new User();
    user.setName(request.getName());
    user.setEmail(request.getEmail());
    user.setPassword(passwordEncoder.encode(request.getPassword()));
    user.setRole(UserRole.ROLE_USER);
    user.setCreatedAt(LocalDateTime.now());
    userRepository.save(user);
}
```

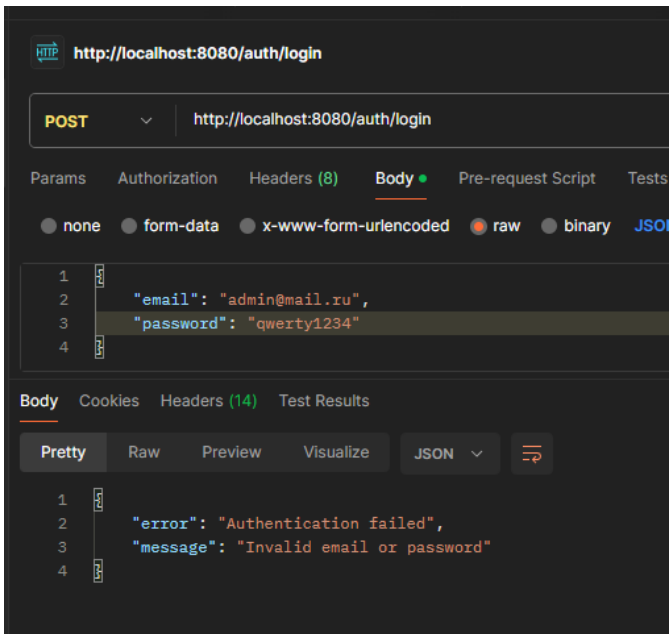
Элемент 8 — Код для проверки уникального email

В контроллере JwtAuthController была добавлена обработка ошибок аутентификации. В листинге под номером 9 представлен метод login(), который перехватывает BadCredentialsException и возвращает статус 401 Unauthorized с понятным сообщением об ошибке. Это улучшает пользовательский опыт, так как клиент получает информативный ответ вместо стандартной серверной ошибки.

```
@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class JwtAuthController {
    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody @Valid LoginRequest request)
{
        try {
            authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(
                            request.getEmail(),
                            request.getPassword()
                    )
            );
            String token = jwtService.generateToken(request.getEmail());
            Map<String, String> response = new HashMap<>();
            response.put("token", token);
            response.put("message", "Login successful");
            return ResponseEntity.ok(response);
        }
        catch (BadCredentialsException e){
            Map<String, String> error = new HashMap<>();
            error.put("error", "Authentication failed");
            error.put("message", "Invalid email or password");
            return
ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
        }
        catch (Exception e) {
            // Другие ошибки аутентификации
            Map<String, String> error = new HashMap<>();
            error.put("error", "Authentication failed");
            error.put("message", "Unable to authenticate user");
            return
ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
        }
    }
}
```

Элемент 9 — Код для обработки ошибок о логине и пароле

На изображении 7 (в нумерации документа — элемент 10) показана проверка обработки ошибок аутентификации. При попытке входа с неверным паролем сервер возвращает статус 401 Unauthorized и сообщение "Invalid email or password". Это подтверждает корректную работу блока try-catch в методе login().



Элемент 10 — Ошибка при неверном логине или пароле

Для устранения дублирования кода преобразования сущности User в UserDto был создан отдельный класс UserMapper. В листинге под номером 11 представлен его код. Маппер содержит методы toDto() и toEntity(), которые выполняют преобразование в обоих направлениях. Это улучшает читаемость кода и упрощает его поддержку, так как логика маппинга вынесена в отдельный компонент.

```
@Component
public class UserMapper {
    public UserDto toDto(User user) {
        if (user == null) {
            return null;
        }
        return UserDto.builder()
                .name(user.getName())
                .email(user.getEmail())
                .phone(user.getPhone())
                .deviceToken(user.getDeviceToken())
                .telegramChatId(user.getTelegramChatId())
                .createdAt(user.getCreatedAt())
                .build();
    }
    public User toEntity(UserDto userDto) {
        if (userDto == null) {
            return null;
        }
        User user = new User();
        user.setName(userDto.getName());
        user.setEmail(userDto.getEmail());
        user.setPhone(userDto.getPhone());
```

```
        user.setDeviceToken(userDto.getDeviceToken());
        user.setTelegramChatId(userDto.getTelegramChatId());
        user.setCreatedAt(userDto.getCreatedAt());
        return user;
    }
}
```

Элемент 11 — UserMapper

После создания UserMapper сервис UserService был обновлён для использования этого компонента. В листинге под номером 12 представлен код обновлённого сервиса. Теперь методы сервиса используют userMapper.toEntity() и userMapper.toDto() вместо ручного преобразования. Это делает код более чистым и соответствует принципу единственной ответственности.

```
package org.example.service;
import lombok.RequiredArgsConstructor;
import org.example.model.dto.UserDto;
import org.example.model.entity.User;
import org.example.model.mapper.UserMapper;
import org.example.repository.UserRepository;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository userRepository;
    private final UserMapper userMapper;

    public UserDto createUser(UserDto request) {
        User user = userMapper.toEntity(request);
        user.setCreatedAt(LocalDateTime.now());
        User savedUser = userRepository.save(user);
        return userMapper.toDto(savedUser);
    }
    public List<UserDto> getAllUsers() {
        return userRepository.findAll()
                .stream()
                .map(userMapper::toDto)
                .collect(Collectors.toList());
    }
    public UserDto getUserById(Long id) {
        User user = userRepository.findById(id).orElseThrow();
        return userMapper.toDto(user);
    }
    public UserDto updateUser(Long id, UserDto request) {
        User user = userRepository.findById(id)
```

```
                .orElseThrow(() -> new RuntimeException("User not found with
id: " + id));

        user.setName(request.getName());
        user.setEmail(request.getEmail());
        user.setPhone(request.getPhone());
        user.setDeviceToken(request.getDeviceToken());
        user.setTelegramChatId(request.getTelegramChatId());

        User updatedUser = userRepository.save(user);
        return userMapper.toDto(updatedUser);
    }
    public void deleteUser(Long id) {
        User user = userRepository.findById(id).orElseThrow();
        userRepository.delete(user);
    }
}
```

Элемент 12 — UserService после вынесения UserMapper

В листинге под номером 13 представлен полный код JwtService с реализованными методами extractUsername() и isTokenExpired(). Метод parseJwt() используется для парсинга токена и получения claims. extractUsername() извлекает subject (email пользователя) из токена. isTokenExpired() проверяет, не истёк ли срок действия токена, сравнивая дату expiration с текущим временем. Метод isTokenValid() выполняет комплексную проверку: сверяет username из токена с данными пользователя и проверяет срок действия.

```
package org.example.security;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jws;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

import static org.hibernate.validator.internal.util.Contracts.assertNotNull;

@Service
public class JwtService {
    private final String secret =
            "verySecretKeyForJwtTokenVerySecretKey12345";

    public String generateToken(String username) {
        SecretKey key =
```

```
                Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        return Jwts.builder()
                .subject(username)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + 1000 * 60
                        * 60))
                .signWith(key, SignatureAlgorithm.HS256)
                .compact();
    }

    public Jws<Claims> parseJwt(String token) {
        SecretKey key =
                Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        Jws<Claims> jwt = Jwts.parser()
                .setSigningKey(key)
                .build()
                .parseClaimsJws(token);
        return jwt;
    }

    public String extractUsername(String token) {
        Jws<Claims> claims = parseJwt(token);
        return claims.getBody().getSubject();
    }

    public boolean isTokenExpired(String token) {
        try {
            Jws<Claims> claims = parseJwt(token);
            Date expiration = claims.getBody().getExpiration();
            return expiration.before(new Date()); // true если просрочен
        } catch (Exception e) {
            return true; // Если ошибка, считаем токен просроченным
        }
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        try {
            String username = extractUsername(token);
            return username.equals(userDetails.getUsername()) &&
!isTokenExpired(token);
        } catch (Exception e) {
            return false;
        }
    }
}
```

Элемент 13 — JwtService с реализованным extractUsername и проверской срока действия токена

В листинге под номером 14 представлена финальная конфигурация безопасности. Ключевые изменения: sessionCreationPolicy установлен в STATELESS, что полностью отключает создание HTTP-сессий. JWT-фильтр добавлен через addFilterBefore() перед UsernamePasswordAuthenticationFilter.

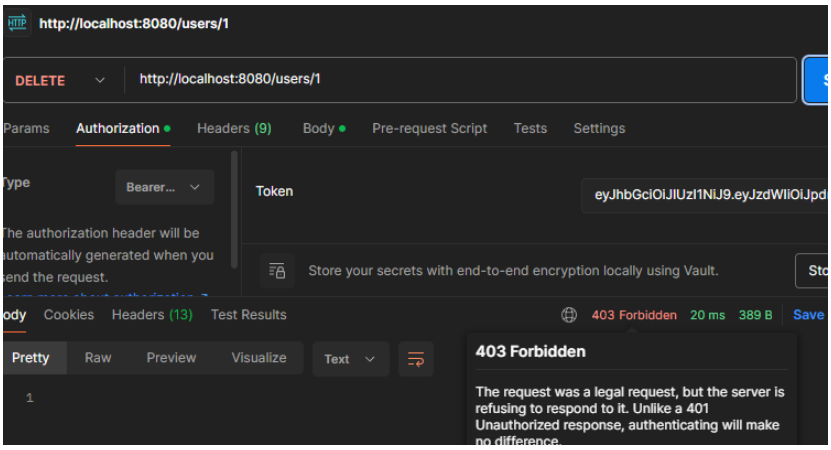
Form login и HTTP Basic отключены, так как вся аутентификация теперь происходит через JWT.

```
@Configuration
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final CustomUserDetailsService customUserDetailsService;
    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(customUserDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }
    @Bean
    public AuthenticationManager
    authenticationManager(AuthenticationConfiguration configuration) throws
            Exception {
        return configuration.getAuthenticationManager();
    }
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/auth/**").permitAll()
                        .requestMatchers("/admin/**").hasRole("ADMIN")
                        .requestMatchers("/users/**").hasAnyRole("USER",
                                "ADMIN")

.requestMatchers("/notifications/**").hasAnyRole("USER", "ADMIN")
                                .anyRequest().authenticated()
                        )
                        .sessionManagement(session -> session

.sessionCreationPolicy(SessionCreationPolicy.STATELESS) // TODO STATELESS for
JWT, IF_REQUIRED for basic
                        )
                        .authenticationProvider(authenticationProvider());
//                        .formLogin(form -> form.disable())
//                        .httpBasic(httpBasic -> {});

        http.addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class)
                .formLogin(form -> form.disable())
                .httpBasic(httpBasic -> httpBasic.disable());;

        return http.build();
```

```
    }
}
```

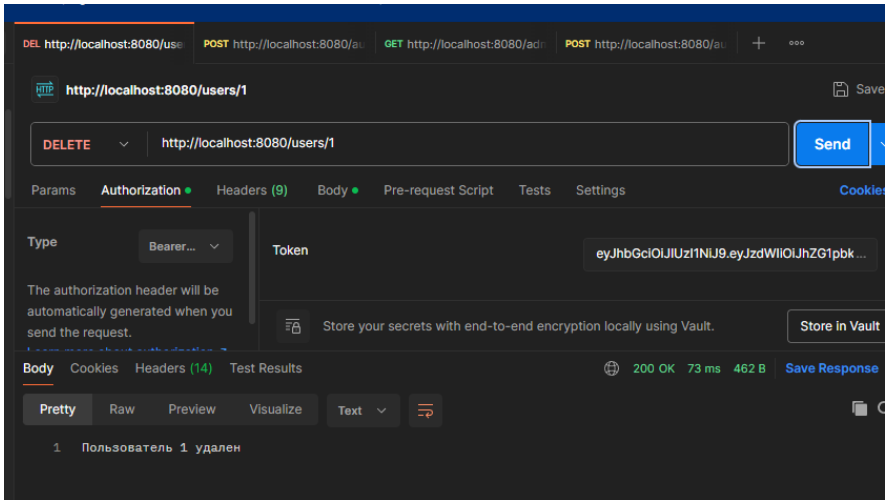
Элемент 14 — SecurityConfig с полной настройкой для JWT.

Для проверки работы @PreAuthorize("hasRole('ADMIN')") была выполнена попытка удаления пользователя от лица обычного пользователя. На изображении 8 показано, что сервер возвращает ошибку 403 Forbidden. Это подтверждает, что аннотация @PreAuthorize корректно защищает метод и не позволяет пользователям без роли ADMIN выполнять операцию удаления.



Элемент 15 — Отсутствие прав обычного пользователя для удаления записей

После аутентификации под администратором был выполнен DELETE-запрос к /users/{id}. На изображении 9 показано, что администратор успешно удаляет пользователя из системы. Сервер возвращает сообщение "Пользователь X удален". Это демонстрирует, что администратор обладает всеми необходимыми правами для выполнения критических операций.



Элемент 16 — Успешное удаления записей с правами ADMIN

В листинге под номером 17 представлен код контроллера UserController, где метод deleteUser() защищён аннотацией @PreAuthorize("hasRole('ADMIN')"). Это декларативный способ защиты методов на уровне контроллера. Аннотация гарантирует, что только пользователи с ролью ADMIN могут вызвать этот метод. Если пользователь не имеет соответствующей роли, Spring Security автоматически возвращает ошибку 403 без выполнения тела метода.

```
@DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public String deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
        return String.format("Пользователь %s удален", id);
    }
}
```

Элемент 17 — Код для удаления записей только с правами ADMIN

Заключение

В ходе выполнения лабораторной работы были успешно освоены ключевые аспекты обеспечения безопасности в Spring Boot-приложении с использованием Spring Security. На примере системы уведомлений, разработанной в лабораторной работе №4, была наглядно продемонстрирована интеграция механизмов аутентификации и авторизации.

Была расширена сущность User: добавлены поля password и role, что позволило хранить учётные данные и права доступа непосредственно в базе данных PostgreSQL. С помощью BCryptPasswordEncoder реализовано надёжное хранение паролей в зашифрованном виде. Созданные CustomUserDetails и CustomUserDetailsService обеспечили адаптацию между моделью пользователя приложения и интерфейсом UserDetails, необходимым Spring Security.

Реализована регистрация пользователей (/auth/register) с проверкой уникальности email и автоматическим назначением роли. Добавлен отдельный

endpoint для регистрации администратора. Настроена конфигурация безопасности (SecurityConfig), задающая правила доступа к URL в зависимости от ролей и отключающая CSRF для упрощения тестирования через Postman.

Были изучены и применены на практике два подхода к аутентификации: **HTTP Basic** (stateful, на основе сессий) и **JWT** (stateless). Для JWT реализованы генерация токена при логине, фильтр JwtAuthenticationFilter для извлечения и валидации токена из заголовка Authorization, а также методы extractUsername() и проверки срока действия токена. Настройка SessionCreationPolicy.STATELESS позволила полностью перевести приложение в stateless-режим, что важно для REST API.

Особое внимание было уделено обработке ошибок: при неверных учётных данных возвращается статус 401 Unauthorized, а при недостатке прав – 403 Forbidden. С помощью аннотации @PreAuthorize защищён метод удаления пользователя, что демонстрирует возможности method-level security.

Github: https://github.com/Prototype721/Java